Module 3: Pretraining Data Pipeline

From Raw Documents to Verified Token Batches

BRAIN team

IMT Atlantique · January 2027

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

BRAIN

Lecture map

1. A tokenizer is a model design choice
2. Packing documents into examples
3. Storage and correctness

Guiding question

How do we turn irregular text into dense, reproducible GPU input without changing the learning task?

What should be clear by the end?

1. Explain byte-level BPE as a learned compression code.
2. Reason about vocabulary size, sequence length, and parameter cost together.
3. Pack documents while keeping boundaries explicit.
4. Verify binary storage, sampling, and the next-token shift.

PART 1

A tokenizer is a model design choice

---

1. A tokenizer is a model design choice
2. Packing documents into examples
3. Storage and correctness

From raw documents to a training corpus

1. Extract text and record the source. WARC preserves web records; WET provides extracted text.
2. Apply documented filters for empty, very short, or repetitive text; inspect what they discard.
3. Group duplicates and split by document before fitting tokenizer merges on training data.
4. Tokenize, add document boundaries, pack and store train and validation separately.

A dataset is a set of choices

Filtering and extraction change the data distribution; record the rules and avoid train–validation leakage.

Granularity moves cost between vocabulary and sequence

Unit	Vocabulary	Sequence	Main cost
bytes	256	very long	attention and context
words	millions	short	embeddings and unknowns
subwords	tunable	moderate	learned merge system

Why byte-level BPE?

Every UTF-8 string decomposes into 256 byte values, so the base system has no unknown text. Frequent byte sequences can then become single tokens.

System consequence

Tokenization changes sequence length, attention work, KV-cache size, multilingual efficiency, and the number of embedding parameters.

BPE learns a greedy compression hierarchy

1. Start from byte IDs and reserved special tokens.
2. Count adjacent token pairs in the training corpus.
3. Merge the most frequent pair into one new token.
4. Repeat until the vocabulary reaches its target size.

$$(p_0, p_1) \mapsto v_{\text{new}}, \qquad \text{repr}(v_{\text{new}}) = \text{repr}(p_0) \| \text{repr}(p_1)$$

Encoding new text

Apply learned merges by rank. Earlier learned pairs win when several merges are possible.

Source: Gage, Sennrich et al., GPT-2, and minBPE

A hand-worked merge exposes the algorithm

Corpus: `low low lower`

Round	frequent pair	stream fragment
0	–	`l o w | l o w | l o w e r`
1	`l o`	`lo w | lo w | lo w e r`
2	`lo w`	`low | low | low e r`

What the table hides

Real byte-level training also defines deterministic tie-breaking, special-token policy, and merge boundaries. These are part of the tokenizer artifact.

Pre-tokenization prevents pathological merges

Naive byte stream

Frequent punctuation and whitespace can merge across unrelated words or document structure.

Regex chunks

Letters, numbers, punctuation, contractions, and whitespace form separate regions. BPE merges stay inside each region.

`" the"` and `"the"` may become different learned tokens because leading-space behavior carries information.

Vocabulary size has two opposing effects

Larger vocabulary

fewer tokens per document
lower attention and decode length
better compression for frequent patterns

But also

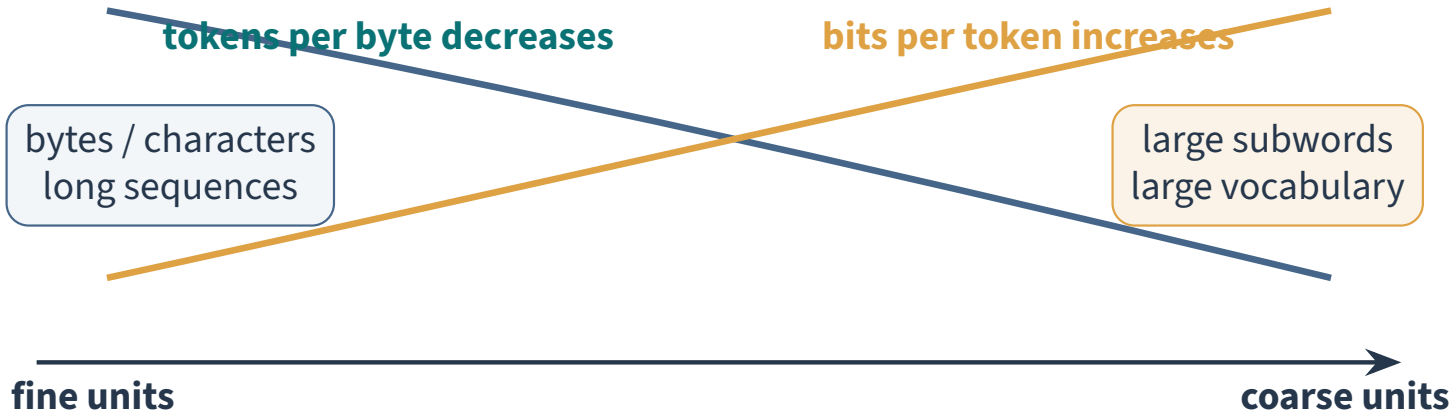
larger embedding and output matrices
rarer tail tokens
more expensive vocabulary softmax

$$\text{embedding parameters} = Vd \qquad \text{compression} = \frac{\text{UTF-8 bytes}}{\text{tokens}}$$

Weight tying avoids paying Vd twice.

Tokenizer efficiency in bits per byte

fidelity gate: $\mathrm{decode}(\mathrm{encode}(s)) = s$ **for every UTF-8 input**



$$\text{bits per byte} = \text{bits per token} \times \text{tokens per byte}$$

Compare tokenizers on the same held-out byte corpus; loss per token alone is not comparable.

Source: MVA Training and Deploying Large-Scale Models, Lecture 1 (2026)

Checkpoint: tokenizer choice

Think, then compare

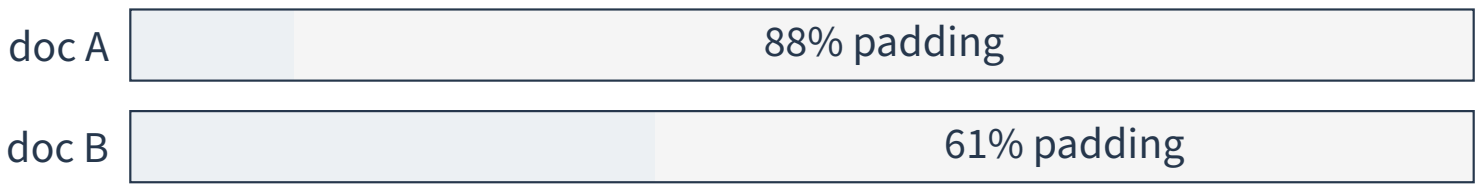
A new tokenizer shortens a representative corpus by 20% but adds 300 million embedding parameters. Which training and serving measurements would decide whether this is a good trade?

PART 2

Packing documents into examples

1. A tokenizer is a model design choice
2. Packing documents into examples
3. Storage and correctness

Padding turns document length variance into wasted FLOPs



Dense packing

Concatenate documents with an explicit end-of-text token, then slice fixed-length training windows from the resulting token stream.

Packing improves utilization but changes attention structure

```
[doc A][EOT][doc B][EOT][doc C]
```

- Every position contributes to the loss.
- Windows can cross a document boundary.
- Standard causal attention lets doc B see doc A.

Two valid policies

Accept cross-document context and record it.

Or use block-diagonal causal masks with document offsets.

Invariant

Train and validation splits must happen at the document level before packing.

The end-of-text token has two roles

1. It preserves a recoverable boundary in a contiguous stream.
2. It teaches the model that one document ended and another may begin.

Special-token policy

Reserve its ID before training merges. Prevent ordinary text from being confused with the special marker. Test round-trip behavior explicitly.

PART 3

Storage and correctness

1. A tokenizer is a model design choice
2. Packing documents into examples
3. Storage and correctness

Binary shards remove tokenization from the hot path

- Pre-tokenize once and store integer IDs contiguously.
- Use `uint16` only when every ID fits below $65{,}536$.
- Open shards with memory mapping and let the OS page cache serve workers.

Storage estimate

10^9 tokens at 2 bytes each occupy 2.0 GB before metadata.

Schema contract

Record dtype, tokenizer hash, token count, and document-boundary convention beside each shard.

One contiguous slice creates inputs and targets

Sample $T+1$ tokens beginning at offset i:

$$x = s[i : i+T] \qquad y = s[i+1 : i+T+1]$$

	0	1	2	$\cdots$	$T-1$
x	s_i	s_{i+1}	s_{i+2}	$\cdots$	s_{i+T-1}
y	s_{i+1}	s_{i+2}	s_{i+3}	$\cdots$	s_{i+T}

Fast correctness check

For every batch, $y[:, :-1]$ must equal $x[:, 1:]$ exactly.

A data pipeline needs end-to-end evidence

1. Encode and decode arbitrary Unicode text losslessly.
2. Verify deterministic token IDs and special-token handling.
3. Measure compression on representative languages and domains.
4. Audit document-level split membership before packing.
5. Saturate a loader benchmark without the GPU waiting for input.

Do not optimize a corrupt stream

Decode sampled training windows back to text. Human inspection often reveals split leakage, repeated boilerplate, or broken boundaries immediately.

Checkpoint: pipeline audit

Think, then compare

A training run is fast, but validation loss is implausibly low. List three document- or token-level checks that distinguish a good model from split contamination.

Key takeaways

1. Byte-level BPE guarantees coverage, while merges trade vocabulary size for shorter sequences.
2. Pre-tokenization and special-token rules are part of the learned tokenizer.
3. Packing removes padding waste but requires an explicit document-boundary policy.
4. Binary shards are useful only when their schema, split, and target shift are verified.